

Kubernetes 企業實戰教育訓練

實作練習題目

了解各站功能與架構後，根據題目進行部署練習

主講人：李旻恩

draw.io

[filebrowser](#)

[flarum](#)

[planka](#)

[onlyoffice](#)

[peertube](#)

[cloudbeaver](#)



Opc Ua Extender

opc UA



USB UltraLock



兩種題型：填空與除錯

題目一 · `manifest-incomplete/`

半成品 YAML 填空

正解被挖掉的欄位標成 `__FILL_ME_n__`，照編號填完再依序 apply。光是 draw.io 就有 20 格。

寫完用 diff 跟 `../manifest/` 對照。先自己推理一輪，再對答案。

題目二 · `manifest-buggy/`

看起來完整，其實藏了錯

整套 apply 下去，用 `describe`、`get -o yaml`、`logs` 找出問題。
draw.io 的四個症狀：

1. Endpoints 永遠是空的 (label)
2. 永遠 0/1 Ready (probe 的 port)
3. HTTPRoute Accepted 卻連不到 (backendRefs)
4. 第二個 Pod 卡在 Pending (配額)

除錯工具箱：看到症狀先查哪裡

症狀	先看哪裡	這門課常見的原因
Pod 卡在 Pending	<code>kubectl describe pod → Events</code>	超過配額、缺 toleration、PVC 未 Bound
Running 但 0/1 Ready	<code>kubectl describe pod → probe</code>	probe 打錯 port、初始延遲太短
Endpoints 是空的	<code>kubectl get endpoints</code>	selector 跟 Pod label 對不上
Gateway 回 503	<code>kubectl describe httproute</code>	backendRefs 不是 Service 的 port
CrashLoopBackOff	<code>kubectl logs --previous</code>	drop ALL 讓降權失敗、設定路徑錯
連線逾時	<code>kubectl exec 測 /dev/tcp</code>	NetworkPolicy 少放行一個方向

驗收標準：四個綠燈才算部署完成

✓ 1

Pod Running 且 Ready

```
kubectl get pod -n <ns>
```

✓ 2

Service 有 endpoints

```
kubectl get endpoints -n  
<ns>
```

✓ 3

**HTTPRoute Accepted
+ ResolvedRefs**

```
kubectl describe  
httproute -n <ns>
```

✓ 4

curl 拿到應用的真實內容

```
curl -k  
https://<product>.nexai.  
org.com
```

單一產品收尾

```
kubectl delete namespace <product>
```

整門課結束：刪掉教學 Gateway 一次收尾

```
kubectl delete -f dev-gateway.yaml
```

第 1 站 · draw.io

無狀態的起點

是開源的圖表繪製工具，可用於繪製架構圖、流程圖、心智圖等，本身是純前端的無狀態 Web 應用（jgraph/drawio 映像），不需要資料庫、不需要任何持久化儲存。



實作：填空（1/3）

manifest-incomplete/ · 共 20 格（第 1/3 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_1__ 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示：這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
__FILL_ME_2__ 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示：跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
__FILL_ME_3__ 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示：Deployment 會開 2 個 replica，每個 request 100m cpu；這個配額至少要能同時容納這兩個 Pod，且要跟同檔案裡的 limits.cpu: "2" 級距相稱（進階提示：太小會讓第二個 Pod 卡在 Pending，可對照 manifest-buggy/ 的除錯題感受這個症狀）
__FILL_ME_4__ 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU（上限） <code>LimitRange.max.cpu</code> 提示：必須 ≥ Deployment 裡容器實際填的 limits.cpu，否則 Pod 會被 LimitRange 擋掉
__FILL_ME_5__ 01-resourcequota-limitrange.yaml	設定單一容器最少要 request 多少記憶體（下限） <code>LimitRange.min.memory</code> 提示：必須 ≤ Deployment 裡容器實際填的 requests.memory，否則 Pod 會被 LimitRange 擋掉（下限擋掉輕量容器是本教材真的踩過的坑）
__FILL_ME_6__ 02-deployment.yaml	這個 Deployment 要開幾個 Pod 副本 <code>spec.replicas</code> 提示：要跟 HPA 的 minReplicas 搭配得上，架構圖上畫了幾個 Pod？

實作：填空 (2/3)

manifest-incomplete/ · 共 20 格 (第 2/3 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_7__ 02-deployment.yaml	填入 draw.io 官方容器映像的名稱與版本 <code>containers[0].image</code> 提示：draw.io 的官方 Docker Hub 映像名稱 + 版本 tag · 架構圖與本檔案上方教學註解都有寫
__FILL_ME_8__ 02-deployment.yaml	填入容器內部應用程式實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示：跟 Service 的 targetPort: http 這個 named port 要對得上
__FILL_ME_9__ 02-deployment.yaml	設定 readiness 探測要打哪個 port <code>readinessProbe.httpGet.port</code> 提示：這裡可以直接引用上面 ports 陣列裡取的 name · 不用重複寫數字 (跟 livenessProbe 那組保持一致寫法)
__FILL_ME_10__ 02-deployment.yaml	設定要捨棄全部 Linux capability <code>securityContext.capabilities.drop</code> 提示：Linux capability 的「全部捨棄」該怎麼寫？ (提示：全大寫的一個字)
__FILL_ME_11__ 03-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示：跟 Deployment 的 template.metadata.labels.app 必須完全一致 · 否則 Service 找不到任何 Endpoint (manifest-buggy/ 的除錯題就是在考這個)
__FILL_ME_12__ 03-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱 · 不是數字
__FILL_ME_13__ 04-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[0].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字

實作：填空 (3/3)

manifest-incomplete/ · 共 20 格 (第 3/3 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_14__ 04-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例：<product>.nexai.org.com · 這個 product 是什麼？
__FILL_ME_15__ 04-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 03-service.yaml 裡 Service 對外開的是幾號 port (不是 containerPort)
__FILL_ME_16__ 05-hpa.yaml	設定 HPA 要控制哪個 Deployment <code>scaleTargetRef.name</code> 提示：HPA 要控制哪個 Deployment？
__FILL_ME_17__ 05-hpa.yaml	設定 HPA 最多能擴到幾個 Pod <code>maxReplicas</code> 提示：架構圖標了 HPA 的擴縮範圍，上限是多少？
__FILL_ME_18__ 05-hpa.yaml	設定 CPU 使用率超過多少百分比要觸發擴容 <code>metrics[0].resource.target.averageUtilization</code> 提示：架構圖上有寫
__FILL_ME_19__ 06-networkpolicy.yaml	設定這條 NetworkPolicy 要保護哪些 Pod <code>allow-ingress-from-gateway.spec.podSelector.matchLabels.app</code> 提示：和 Service/Deployment 用同一組 label
__FILL_ME_20__ 06-networkpolicy.yaml	設定只放行進到容器的哪個 port <code>allow-ingress-from-gateway.spec.ingress[0].ports[0].port</code> 提示：跟 __FILL_ME_8__ 應該是同一個號碼

實作：填空・解答

20 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	drawio
2	00 · label product	drawio
3	01 · quota cpu	"1"
4	01 · max.cpu	"1"
5	01 · min.memory	64Mi
6	02 · replicas	2
7	02 · image	jgraph/drawio:24.7.17
8	02 · containerPort	8080
9	02 · readiness port	http
10	02 · drop	ALL

#	欄位	正解
11	03 · selector.app	drawio
12	03 · targetPort	http
13	04 · parentRefs	dev-gateway
14	04 · hostnames	" drawio.nexai.org.com "
15	04 · backendRefs	80
16	05 · scaleTargetRef	drawio
17	05 · maxReplicas	5
18	05 · averageUtilization	70
19	06 · podSelector	drawio
20	06 · ingress port	8080

實作：藏錯

manifest-buggy/ · 藏了 4 個錯 · 每個檔案最多一個 · 先整套 apply · 再從症狀往回追

1 Endpoints 永遠是空的

Pod 是 Running，透過 Gateway 連線卻得到 503 / connection refused。

先查

```
kubectl get endpoints drawio -n drawio
```

2 永遠 0/1 Ready

Pod Running 但 Ready 一直是 0，Events 出現 probe 失敗。錯的不是 containerPort，是引用它的地方。

先查

```
kubectl describe pod -n drawio
```

3 HTTPRoute Accepted 卻連不到

HTTPRoute 狀態看起來正常，實際流量卻失敗。backendRefs 的號碼該對應 Service 的哪個 port？

先查

```
kubectl describe httproute drawio -n drawio
```

4 第二個 Pod 卡在 Pending

replicas=2 的第二隻一直起不來，Events 看到 exceeded quota。

先查

```
kubectl describe pod -n drawio
```

實作：藏錯・解答

4 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	Endpoints 永遠是空的	03-service.yaml	錯 <code>selector.app: draw</code> 改 <code>selector.app: drawio</code>	selector 要跟 Pod label 完全一致；Service 只認 label。
2	永遠 0/1 Ready	02-deployment.yaml	錯 <code>readinessProbe port: 8081</code> 改 <code>readinessProbe port: http</code>	probe 直接引用 named port，containerPort 改了也不會跟著錯。
3	HTTPRoute Accepted 卻連不到	04-httproute.yaml	錯 <code>backendRefs port: 8080</code> 改 <code>backendRefs port: 80</code>	backendRefs 填 Service 的 port，不是 containerPort。
4	第二個 Pod 卡在 Pending	01-resourcequota-limitrange.yaml	錯 <code>requests.cpu: "100m"</code> 改 <code>requests.cpu: "1"</code>	2 個 Pod × 100m 就超過；HPA 最多 5 × 100m = 500m，抓 1 留餘裕。

第 2 站 · filebrowser

RWX vs RWO，有狀態

是一個網頁版檔案管理器，提供瀏覽、上傳、下載、預覽，並整合 onlyoffice 做線上編輯 Word / Excel / PPT。

(實際跑的映像是社群 fork FileBrowser Quantum，gtstef/filebrowser:1.5.6-stable)



實作：填空（ 1/4 ）

manifest-incomplete/ · 共 25 格（ 第 1/4 頁 ） · 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_1__ 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示：這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
__FILL_ME_2__ 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示：跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
__FILL_ME_3__ 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時申請多少 PVC 總儲存空間 <code>ResourceQuota.hard.requests.storage</code> 提示：看 02-pvc.yaml 兩顆 PVC 各申請多少 storage，加總後再留一點擴容餘裕（詳見上面「資源該給多少」章節）
__FILL_ME_4__ 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少記憶體（上限） <code>LimitRange.max.memory</code> 提示：必須 ≥ Deployment 容器實際填的 limits.memory，否則 Pod 會被 LimitRange 擋掉
__FILL_ME_5__ 02-pvc.yaml	filebrowser-srv 這顆 PVC 要能被多個 Pod 同時掛載讀寫，該選哪一種存取模式 <code>filebrowser-srv.spec.accessModes[0]</code> 提示：這顆 PVC 放的是使用者實際檔案，將來要支援多 Pod 同時讀寫；本產品說明段已經寫了答案是哪一種模式
__FILL_ME_6__ 02-pvc.yaml	filebrowser-srv 該接哪一個 StorageClass <code>filebrowser-srv.spec.storageClassName</code> 提示：要支援上一題選的存取模式，叢集裡哪個 StorageClass 是 CephFS（檔案系統型、支援多寫）？

實作：填空（2/4）

manifest-incomplete/ · 共 25 格（第 2/4 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_7__ 02-pvc.yaml	filebrowser-db 放的是 sqlite 資料庫，只能有一個行程寫入，該選哪一種存取模式 <code>filebrowser-db.spec.accessModes[0]</code> 提示：跟上面 filebrowser-srv 的選擇正好相反，為什麼？
__FILL_ME_8__ 02-pvc.yaml	filebrowser-db 該接哪一個 StorageClass <code>filebrowser-db.spec.storageClassName</code> 提示：要支援上一題選的存取模式，叢集裡哪個 StorageClass 是 Ceph RBD（區塊儲存、單寫）？
__FILL_ME_9__ 03-configmap.yaml	關掉啟動時對外檢查新版本的功能 <code>data."config.yaml".server.disableUpdateCheck</code> 提示：檔案上方的教學註解完整解釋了這個踩過的坑（開機卡 31 秒），該填 true 還是 false？
__FILL_ME_10__ 03-configmap.yaml	設定 filebrowser 後端要用叢集內部哪個網址呼叫自己（給 OnlyOffice Document Server 回頭抓檔案用） <code>data."config.yaml".server.internalUrl</code> 提示：格式是 <code>http://<Service 名稱>.<namespace>.svc.cluster.local</code> ，對照 05-service.yaml 跟 00-namespace.yaml 填的值
__FILL_ME_11__ 03-configmap.yaml	設定檔案來源要對應到容器裡的哪個掛載路徑 <code>data."config.yaml".server.sources[0].path</code> 提示：看 04-deployment.yaml 的 volumeMounts，filebrowser-srv 這顆 PVC 掛在容器內的哪個路徑？
__FILL_ME_12__ 04-deployment.yaml	這個 Deployment 要開幾個 Pod 副本 <code>spec.replicas</code> 提示：sqlite 資料庫不支援多個行程同時寫入，這個產品說明段已經講了答案

實作：填空 (3/4)

manifest-incomplete/ · 共 25 格 (第 3/4 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_13__</u> 04-deployment.yaml	設定滾動更新策略要用哪一種 <code>spec.strategy.type</code> 提示： 跟 draw.io 用的 RollingUpdate 不一樣——因為 RWO 的 filebrowser-db 同時只能被一個節點掛載，若新舊 Pod 同時存在會搶不到 volume，該用哪個策略先把舊 Pod 整個砍掉，再建立新的？
<u>__FILL_ME_14__</u> 04-deployment.yaml	填入 FileBrowser Quantum 官方穩定線容器映像的名稱與版本 <code>containers[0].image</code> 提示： ../README.md 開頭已經寫了目前用的正式版映像 tag
<u>__FILL_ME_15__</u> 04-deployment.yaml	填入容器內部應用程式實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示： 跟 Service 的 targetPort 這個 named port、還有 ConfigMap 裡 server.port 要對得上
<u>__FILL_ME_16__</u> 04-deployment.yaml	設定 readiness 探測要打哪個 port <code>readinessProbe.httpGet.port</code> 提示： 可以直接引用上面 ports 陣列裡取的 name，不用重複寫數字（跟 livenessProbe 那組保持一致寫法）
<u>__FILL_ME_17__</u> 04-deployment.yaml	設定要捨棄全部 Linux capability <code>securityContext.capabilities.drop</code> 提示： Linux capability 的「全部捨棄」該怎麼寫？（提示：全大寫的一個字）
<u>__FILL_ME_18__</u> 05-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示： 跟 Deployment 的 template.metadata.labels.app 必須完全一致，否則 Service 找不到任何 Endpoint（manifest-buggy/ 的除錯題就是在考類似的坑）

實作：填空（4/4）

manifest-incomplete/ · 共 25 格（第 4/4 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_19__ 05-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱，不是數字
__FILL_ME_20__ 06-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[0].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字
__FILL_ME_21__ 06-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例：<product>.nexai.org.com，這個 product 是什麼？
__FILL_ME_22__ 06-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 05-service.yaml 裡 Service 對外開的是幾號 port（不是 containerPort）
__FILL_ME_23__ 07-networkpolicy.yaml	設定 allow-ingress-from-gateway 這條規則要保護哪些 Pod <code>spec.podSelector.matchLabels.app</code> 提示：和 Service/Deployment 用同一組 label
__FILL_ME_24__ 07-networkpolicy.yaml	設定只放行進到容器的哪個 port <code>spec.ingress[0].ports[0].port</code> 提示：跟 __FILL_ME_15__ 應該是同一個號碼
__FILL_ME_25__ 07-networkpolicy.yaml	設定 allow-egress-filebrowser-to-onlyoffice 這條 egress 規則要放行到哪個 namespace <code>spec.egress[0].to[0].namespaceSelector.matchLabels."kubernetes.io/metadata.name"</code> 提示：filebrowser 要主動呼叫哪個產品的 Document Server 才能做 OnlyOffice 整合？

實作：填空・解答

25 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	filebrowser
2	00 · label product	filebrowser
3	01 · quota storage	20Gi
4	01 · max.memory	512Mi
5	02 · accessModes	ReadWriteMany
6	02 · storageClass	rook-cephfs
7	02 · accessModes	ReadWriteOnce
8	02 · storageClass	rook-ceph-block
9	03 · disableUpdateCheck	true
10	03 · internalUrl	http://filebrowser.filebrowser.svc.cluster.local
11	03 · sources.path	/srv
12	04 · replicas	1
13	04 · strategy.type	Recreate

#	欄位	正解
14	04 · image	gtstef/filebrowser:1.5.6-stable
15	04 · containerPort	80
16	04 · readiness port	http
17	04 · drop	ALL
18	05 · selector.app	filebrowser
19	05 · targetPort	http
20	06 · parentRefs	dev-gateway
21	06 · hostnames	"filebrowser.nexai.org.com"
22	06 · backendRefs	80
23	07 · podSelector	filebrowser
24	07 · ingress port	80
25	07 · namespaceSelector	onlyoffice

實作：藏錯

manifest-buggy/ · 藏了 4 個錯 · 每個檔案最多一個 · 先整套 apply · 再從症狀往回追

1 PVC 一直卡在 Pending

其中一顆 PVC 遲遲不 Bound · Events 提到 provisioning / access mode 。

先查

```
kubectl describe pvc -n filebrowser
```

2 Endpoints 永遠是空的

Pod Running · 透過 Gateway 連線得到 503 / connection refused 。

跟 label 有關 。

先查

```
kubectl get endpoints filebrowser -n filebrowser
```

3 找不到設定檔

Pod 卡在 ContainerCreating · 或一啟動就報錯找不到設定檔 / 某個 key 。

先查

```
kubectl describe pod -n filebrowser
```

4 下一次更新才卡住

第一次 apply 完全正常；之後改 image tag 或任何欄位，新 Pod 就卡住 。

先查

```
kubectl rollout status deploy/filebrowser -n filebrowser
```

實作：藏錯・解答

4 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	PVC 一直卡在 Pending	02-pvc.yaml	錯 <code>filebrowser-srv: rook-ceph-block</code> 改 <code>filebrowser-srv: rook-cephfs</code>	RWX 需要檔案系統型的 CephFS；RBD 區塊儲存不支援多節點寫入。
2	Endpoints 永遠是空的	05-service.yaml	錯 <code>selector.app: file-browser</code> 改 <code>selector.app: filebrowser</code>	selector 必須和 Pod label 一字不差。
3	找不到設定檔	03-configmap.yaml	錯 <code>data key: config.yml</code> 改 <code>data key: config.yaml</code>	Deployment 用 <code>subPath: config.yaml</code> 掛載，ConfigMap 的 key 要同名。
4	下一次更新才卡住	04-deployment.yaml	錯 <code>strategy.type: RollingUpdate</code> 改 <code>strategy.type: Recreate</code>	RWO 的 <code>filebrowser-db</code> 只能掛一個節點，舊 Pod 要先完全終止釋放 volume。

延伸：踩坑案例

容器 FATAL：找不到設定檔

原因

image 的 entrypoint 不轉發 -c 參數，只認官方預設路徑。

解法

不跟 entrypoint 對抗：用 subPath 把 ConfigMap 疊到預設路徑的 config.yaml。

編輯器顯示「目前無法存取該檔案」

原因

document.url 用了瀏覽器網址；onlyoffice 回頭抓檔案的方向也沒放行。

解法

設定 server.internalUrl，並補上 onlyoffice → filebrowser 的 NetworkPolicy。

每次開機卡 31 秒

原因

啟動時同步檢查 GitHub 新版本，連線被 NetworkPolicy 悄悄丟棄。

解法

server.disableUpdateCheck: true，開機時間降到 2 秒內。

第 3 站 ·

App 與 MySQL 住同一個 namespace

是極簡、輕量且現代化的開源討論區/論壇軟體。([mondedie/flarum](#) 映像)



實作：填空（1/5）

manifest-incomplete/ · 共 35 格（第 1/5 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_1__ 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示：這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
__FILL_ME_2__ 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示：跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
__FILL_ME_3__ 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示：flarum 與 mysql 兩個容器的 requests.cpu 加總是多少？這個配額至少要能同時容納兩者，並留一點餘裕（上方「資源該給多少」章節有完整算法）
__FILL_ME_4__ 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU（上限） <code>LimitRange.max.cpu</code> 提示：這個 namespace 裡兩個工作負載（flarum Deployment、mysql StatefulSet）各自的 limits.cpu 哪個比較高？max 至少要蓋過那一個，否則那個 Pod 會建立不起來
__FILL_ME_5__ 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少記憶體（上限） <code>LimitRange.max.memory</code> 提示：同上，但這次兩個工作負載的 limits.memory 剛好一樣高，max 要留多少緩衝？
__FILL_ME_6__ 02-mysql-secret.yaml	幫這個 Secret 取名字 <code>metadata.name</code> 提示：07 號檔案的 flarum Deployment 會用 secretKeyRef 指定這個名字去讀 MYSQL_PASSWORD，兩邊要對得上

實作：填空 (2/5)

manifest-incomplete/ · 共 35 格 (第 2/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_7__</u> 03-mysql-service.yaml	設定這個 Service 要變成 Headless (無 ClusterIP) <code>spec.clusterIP</code> 提示：StatefulSet 要搭配哪種特殊值的 Service，才能讓每個 Pod 拿到穩定的 DNS 名稱？
<u>__FILL_ME_8__</u> 04-mysql-statefulset.yaml	設定這個 StatefulSet 要用哪個 Headless Service 提供網路身分 <code>spec.serviceName</code> 提示：要跟 03-mysql-service.yaml 建立的 Service 名稱一致
<u>__FILL_ME_9__</u> 04-mysql-statefulset.yaml	設定容器要從哪個 Secret 讀取一整組環境變數 <code>envFrom[0].secretRef.name</code> 提示：跟 02-mysql-secret.yaml 建立的 Secret 名稱一致 (也就是 __FILL_ME_6__ 的答案)
<u>__FILL_ME_10__</u> 04-mysql-statefulset.yaml	設定 livenessProbe 要等多久才開始檢查 <code>livenessProbe.initialDelaySeconds</code> 提示：上面教學註解已經寫了實測數字：mysql 第一次 initdb 在這座叢集要跑多久？抓太短會怎樣？
<u>__FILL_ME_11__</u> 04-mysql-statefulset.yaml	設定 PVC 要用哪個 StorageClass <code>volumeClaimTemplates[0].spec.storageClassName</code> 提示：mysql 資料庫是單一 Pod 寫入，該選 RWO 還是 RWX 的 StorageClass？這座叢集的 RWO StorageClass 叫什麼？
<u>__FILL_ME_12__</u> 05-flarum-secret.yaml	幫這個 Secret 取名字 <code>metadata.name</code> 提示：07 號檔案的 flarum Deployment 會用 secretKeyRef 指定這個名字去讀 FLARUM_ADMIN_PASS
<u>__FILL_ME_13__</u> 05-flarum-secret.yaml	設定 Flarum 後台管理員的登入密碼 <code>stringData.FLARUM_ADMIN_PASS</code> 提示：只是教學用途，跟 ../flarum/README.md 寫的預設密碼一致即可

實作：填空（3/5）

manifest-incomplete/ · 共 35 格（第 3/5 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_14__</u> 06-flarum-pvc.yaml	設定 flarum-storage 這顆 PVC 的存取模式 <code>accessModes[0]</code> (flarum-storage) 提示：只有一個 replica 在寫這些執行期資料 (log/cache/session)，需要多個節點同時掛載嗎？
<u>__FILL_ME_15__</u> 06-flarum-pvc.yaml	設定 flarum-storage 這顆 PVC 要用哪個 StorageClass <code>storageClassName</code> (flarum-storage) 提示：呼應上一題的存取模式，這座叢集哪個 StorageClass 對應 RWO？
<u>__FILL_ME_16__</u> 06-flarum-pvc.yaml	設定 flarum-assets 這顆 PVC 的存取模式 <code>accessModes[0]</code> (flarum-assets) 提示：上傳的頭像/附加檔未來可能要給多個 Pod 同時讀寫，需要哪種存取模式？
<u>__FILL_ME_17__</u> 06-flarum-pvc.yaml	設定 flarum-assets 這顆 PVC 要用哪個 StorageClass <code>storageClassName</code> (flarum-assets) 提示：呼應上一題，這座叢集哪個 StorageClass 支援多寫者 (RWX)？
<u>__FILL_ME_18__</u> 07-flarum-deployment.yaml	決定要不要關閉 K8s 自動注入的 Service 環境變數 <code>spec.template.spec.enableServiceLinks</code> 提示：上面教學註解解釋了 FLARUM_PORT 撞名的坑，這裡該填 true 還是 false？
<u>__FILL_ME_19__</u> 07-flarum-deployment.yaml	設定 initContainer 要把目錄的擁有者改成哪個 uid:gid <code>initContainers[0].command</code> (chown -R 的目標) 提示：要跟主容器實際執行的使用者一致，才不會主容器沒有寫入權限（提示：同檔案 fsGroup 那行寫的號碼，兩處要一致）
<u>__FILL_ME_20__</u> 07-flarum-deployment.yaml	設定要連線到哪個資料庫主機 <code>env[DB_HOST].value</code> 提示：同 namespace 裡 mysql 的 Service 叫什麼名字？不需要完整 FQDN

實作：填空（4/5）

manifest-incomplete/ · 共 35 格（第 4/5 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_21__</u> 07-flarum-deployment.yaml	設定要從哪個 Secret 讀取資料庫密碼 <code>env[DB_PASS].valueFrom.secretKeyRef.name</code> 提示：這是全課程唯一一個「同 namespace 兩個 Secret 互相引用」的例子，密碼實際存在哪個 Secret 物件裡？
<u>__FILL_ME_22__</u> 07-flarum-deployment.yaml	設定要從哪個 Secret 讀取後台管理員密碼 <code>env[FLARUM_ADMIN_PASS].valueFrom.secretKeyRef.name</code> 提示：跟上一題是不同的 Secret，答案在 05 號檔案
<u>__FILL_ME_23__</u> 07-flarum-deployment.yaml	設定 livenessProbe 要等多久才開始檢查 <code>livenessProbe.initialDelaySeconds</code> 提示：Flarum 第一次啟動要實際跑完安裝流程（連 MySQL、建表），這個延遲該抓多寬鬆？
<u>__FILL_ME_24__</u> 07-flarum-deployment.yaml	設定 flarum-storage 這顆 PVC 要掛到容器內的哪個路徑 <code>volumeMounts[storage].mountPath</code> 提示：只掛「資料類」子目錄，不要整個 /flarum/app（上面教學註解解釋了為什麼），這裡該填哪個路徑？
<u>__FILL_ME_25__</u> 07-flarum-deployment.yaml	設定 flarum-assets 這顆 PVC 要掛到容器內的哪個路徑 <code>volumeMounts[assets].mountPath</code> 提示：同上，這次是給使用者上傳頭像/附加檔用的子目錄
<u>__FILL_ME_26__</u> 08-flarum-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示：跟 07-flarum-deployment.yaml 的 template.metadata.labels.app 必須完全一致
<u>__FILL_ME_27__</u> 08-flarum-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱，不是數字

實作：填空 (5/5)

manifest-incomplete/ · 共 35 格 (第 5/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_28__ 09-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[*].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字 (http 、 https 兩個 parentRefs 都是同一個答案)
__FILL_ME_29__ 09-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例：<product>.nexai.org.com · 這個 product 是什麼？且要跟 07 號檔案的 FORUM_URL 一致
__FILL_ME_30__ 09-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 08-flarum-service.yaml 裡 Service 對外開的是幾號 port (不是 containerPort)
__FILL_ME_31__ 10-networkpolicy.yaml	設定 allow-ingress-to-mysql-from-flarum 這條規則要保護哪些 Pod <code>podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你保護的對象是誰
__FILL_ME_32__ 10-networkpolicy.yaml	設定只放行哪個標籤的 Pod 連進 mysql <code>ingress[0].from[0].podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你允許誰連進來
__FILL_ME_33__ 10-networkpolicy.yaml	設定只放行進到 mysql 的哪個 port <code>ingress[0].ports[0].port</code> 提示：mysql 對外服務的是哪個號碼？跟 03-mysql-service.yaml 的 port 一致
__FILL_ME_34__ 10-networkpolicy.yaml	設定 allow-egress-flarum-to-mysql 這條規則要限制哪些 Pod 的對外流量 <code>podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你限制的對象是誰
__FILL_ME_35__ 10-networkpolicy.yaml	設定這些 Pod 的對外流量只能連到哪個標籤的 Pod <code>egress[0].to[0].podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你允許連到誰

實作：填空・解答

35 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	flarum
2	00 · label product	flarum
3	01 · quota cpu	"750m"
4	01 · max.cpu	"1"
5	01 · max.memory	2Gi
6	02 · Secret name	mysql-secret
7	03 · clusterIP	None
8	04 · serviceName	mysql
9	04 · secretRef	mysql-secret
10	04 · liveness delay	60
11	04 · storageClass	rook-ceph-block
12	05 · Secret name	flarum-secret

#	欄位	正解
13	05 · FLARUM_ADMIN_PASS	"TrainingAdmin123!"
14	06 · accessModes	ReadWriteOnce
15	06 · storageClass	rook-ceph-block
16	06 · accessModes	ReadWriteMany
17	06 · storageClass	rook-cephfs
18	07 · enableServiceLinks	false
19	07 · chown uid:gid	991:991
20	07 · DB_HOST	"mysql"
21	07 · secretRef DB_PASS	mysql-secret
22	07 · secretRef ADMIN_PASS	flarum-secret
23	07 · liveness delay	60
24	07 · mountPath	/flarum/app/storage

#	欄位	正解
25	07 · mountPath	/flarum/app/public/assets
26	08 · selector.app	flarum
27	08 · targetPort	http
28	09 · parentRefs	dev-gateway
29	09 · hostnames	"flarum.nexai.org.com"
30	09 · backendRefs	80
31	10 · podSelector	mysql
32	10 · from podSelector	flarum
33	10 · ingress port	3306
34	10 · podSelector	flarum
35	10 · to podSelector	mysql

實作：藏錯

manifest-buggy/ · 藏了 5 個錯 · 每個檔案最多一個 · 先整套 apply，再從症狀往回追

1 mysql 過一陣子就重啟

剛部署看起來正常，之後
CrashLoopBackOff，密碼對也連不進去，
只能砍 PVC 重來。

先查

```
kubectl describe pod mysql-0 -n flarum
```

2 flarum 連 DB 認證失敗

flarum 起不來，看得到資料庫連線 / 認證錯誤，但 mysql 本身是健康的。

先查

```
kubectl logs deploy/flarum -n flarum
```

3 收到沒設定過的埠號

flarum 啟動後很快崩潰，錯誤訊息像是監聽的 port 設定對不上。

先查

```
kubectl logs deploy/flarum -n flarum
```

4 程式檔案不見了

flarum 一直不 Ready 或馬上出錯，像是程式碼不見、預期的路徑是空的。

先查

```
kubectl describe pod -n flarum
```

5 套用 NetworkPolicy 後逾時

兩邊 Pod 都健康，套用 10-networkpolicy 之後 flarum 連 mysql 卻 timeout。

先查

```
nc -zv mysql 3306 (在 flarum Pod 內)
```

除錯順序

1. get events：最近發生了什麼
2. describe / logs：找第一手線索
3. exec 進 Pod 測 DNS 與連線
4. 最後才 diff ../manifest/ 對答案

實作：藏錯・解答

5 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	mysql 過一陣子就重啟	04-mysql-statefulset.yaml	錯 <code>liveness initialDelaySeconds: 20</code> 改 <code>liveness initialDelaySeconds: 60</code>	第一次 initdb 還沒跑完就被 livenessProbe 砍掉，資料目錄初始化到一半壞掉。
2	flarum 連 DB 認證失敗	07-flarum-deployment.yaml	錯 <code>DB_PASS secretKeyRef: flarum-secret</code> 改 <code>DB_PASS secretKeyRef: mysql-secret</code>	MYSQL_PASSWORD 存在 mysql-secret；flarum-secret 只有管理員密碼。
3	收到沒設定過的埠號	07-flarum-deployment.yaml	錯 (缺) <code>enableServiceLinks</code> 改 <code>enableServiceLinks: false</code>	Service 叫 flarum，K8s 自動注入 FLARUM_PORT=tcp://... 蓋掉應用設定。
4	程式檔案不見了	07-flarum-deployment.yaml	錯 <code>storage mountPath: /flarum/app</code> 改 <code>mountPath: /flarum/app/storage</code>	空 PVC 掛到整個 /flarum/app，把 image 裡的程式蓋掉了。
5	套用 NetworkPolicy 後逾時	10-networkpolicy.yaml	錯 <code>from podSelector app: flarum-app</code> 改 <code>from podSelector app: flarum</code>	放行來源的 label 要對到 flarum Pod 真正的 label。

第 4 站 · plank

同樣架構，換成 PostgreSQL

是開源、輕量的看板式專案管理工具。（ghcr.io/plankanban/plank 映像）



實作：填空（1/5）

manifest-incomplete/ · 共 30 格（第 1/5 頁）· 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_1__</u> 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示： README 標題已經告訴你這個產品叫什麼，之後每個檔案的 namespace: 都要跟它一致
<u>__FILL_ME_2__</u> 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示： 跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
<u>__FILL_ME_3__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示： 這個 namespace 同時跑 postgres (requests.cpu 200m) 跟 planka (requests.cpu 150m) 兩個元件，配額至少要能同時容納兩者加總 (350m)，並留一點手動擴充的餘裕（詳見上方「資源判斷」章節）
<u>__FILL_ME_4__</u> 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU（上限） <code>LimitRange.max.cpu</code> 提示： 必須 $\geq$ namespace 裡「limits.cpu 最大」的那個容器；這個 namespace 裡誰的 limits.cpu 最大，postgres 還是 planka？
<u>__FILL_ME_5__</u> 01-resourcequota-limitrange.yaml	設定單一容器最少要 request 多少記憶體（下限） <code>LimitRange.min.memory</code> 提示： 必須 $\leq$ namespace 裡「requests.memory 最小」的那個容器；這個 namespace 裡誰的 requests.memory 最小？
<u>__FILL_ME_6__</u> 02-postgres-secret.yaml	設定 PostgreSQL 的資料庫密碼 <code>stringData.POSTGRES_PASSWORD</code> 提示： 這個密碼之後會被 05-planka-secret.yaml 的 DATABASE_URL 引用，兩邊必須一致；密碼故意不含 @:/ 這類 URI 保留字元

實作：填空（2/5）

manifest-incomplete/ · 共 30 格（第 2/5 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_7__</u> 03-postgres-service.yaml	設定這個 Service 要不要有自己的 ClusterIP <code>spec.clusterIP</code> 提示：StatefulSet 通常搭配哪一種特殊 Service，讓每個 Pod 有自己穩定的 DNS 名稱，而不是共用一個虛擬 IP？（提示：檔名已經寫了 Headless）
<u>__FILL_ME_8__</u> 03-postgres-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示：跟 04-postgres-statefulset.yaml 的 template.metadata.labels.app 必須完全一致
<u>__FILL_ME_9__</u> 04-postgres-statefulset.yaml	設定這個 StatefulSet 要綁定哪個 Headless Service <code>spec.serviceName</code> 提示：要對到 03-postgres-service.yaml 的 metadata.name
<u>__FILL_ME_10__</u> 04-postgres-statefulset.yaml	設定 PostgreSQL 資料目錄實際要放在掛載點底下的哪個子路徑 <code>env[PGDATA].value</code> 提示：不要直接指到 volumeMounts.mountPath 的根目錄——Ceph RBD 格式化成 ext4 後，掛載點根目錄一開始就有一個資料夾，會讓 postgres 的 initdb 誤判「目錄不是空的」而拒絕啟動，指到子目錄可以避開這個問題
<u>__FILL_ME_11__</u> 04-postgres-statefulset.yaml	設定 liveness 探測要打哪個 port <code>livenessProbe.tcpSocket.port</code> 提示：可以直接引用上面 ports 陣列取的 name，不用重複寫數字 5432
<u>__FILL_ME_12__</u> 04-postgres-statefulset.yaml	設定這顆 PVC 的存取模式 <code>volumeClaimTemplates.spec.accessModes</code> 提示：這顆 PVC 全程只會有 1 個 postgres Pod 掛它，該選單一節點讀寫還是多節點讀寫？

實作：填空 (3/5)

manifest-incomplete/ · 共 30 格 (第 3/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_13__ 04-postgres-statefulset.yaml	設定這顆 PVC 要用叢集裡哪一種 StorageClass <code>volumeClaimTemplates.spec.storageClassName</code> 提示：資料庫類單寫場景，這座叢集提供的哪一種 StorageClass 對應 Ceph RBD (區塊儲存) ？
__FILL_ME_14__ 05-planka-secret.yaml	組出 Planka 連線 PostgreSQL 用的完整連線字串 <code>stringData.DATABASE_URL</code> 提示：格式是 postgresql://<user>:<password>@<host>:<port>/<db> ； user/password/db 要跟 02-postgres-secret.yaml 一致，host 直接用 03-postgres-service.yaml 的 Service 名稱即可 (同 namespace 不需要 FQDN)
__FILL_ME_15__ 06-planka-pvc.yaml	設定這顆 PVC 的存取模式 <code>spec.accessModes</code> 提示：使用者上傳的頭像/附件未來可能要給多個 Pod 同時讀寫，該選哪一種存取模式？ (跟 __FILL_ME_12__ 的 postgres PVC 刻意不同，注意對照)
__FILL_ME_16__ 07-planka-deployment.yaml	填入 Planka 官方容器映像的名稱與版本 <code>containers[0].image</code> 提示：Planka 官方發布在 ghcr.io (不是 Docker Hub) 的映像名稱，本檔案上方教學註解跟 ../README.md 都有提到這個 image 拉取比較慢
__FILL_ME_17__ 07-planka-deployment.yaml	設定對外宣告的服務網址 <code>env[BASE_URL].value</code> 提示：這個值要跟 09-httproue.yaml 的 hostnames 一致，Planka 拿它組前端頁面裡的絕對網址
__FILL_ME_18__ 07-planka-deployment.yaml	填入容器內部應用程式實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示：跟 08-planka-service.yaml 的 targetPort: http 這個 named port 要對得上 (Planka/Sails 預設監聽的 port 號碼比較特別，不是常見的 3000/8080)

實作：填空（4/5）

manifest-incomplete/ · 共 30 格（第 4/5 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_19__</u> 07-planka-deployment.yaml	設定 readiness 探測要打哪個 port readinessProbe.httpGet.port 提示：可以直接引用上面 ports 陣列取的 name，不用重複寫數字（跟 livenessProbe 那組保持一致寫法）
<u>__FILL_ME_20__</u> 07-planka-deployment.yaml	設定要捨棄全部 Linux capability securityContext.capabilities.drop 提示：官方 planka image 是用什麼身分啟動的？（提示：不像 postgres 需要先以 root 啟動再降權，這裡可以放心全部捨棄，寫法跟 draw.io 範例的 __FILL_ME_10__ 一樣）
<u>__FILL_ME_21__</u> 07-planka-deployment.yaml	設定容器內部要把 PVC 掛到哪個路徑 volumeMounts[0].mountPath 提示：Planka（Node.js/Sails 應用）會在這個路徑下自己建立 protected/、private/ 等子目錄，不需要 initContainer 預先建立——路徑本身是 Planka 應用程式碼寫死的慣例路徑
<u>__FILL_ME_22__</u> 08-planka-service.yaml	設定這個 Service 要選中哪些 Pod spec.selector.app 提示：跟 07-planka-deployment.yaml 的 template.metadata.labels.app 必須完全一致，否則 Service 找不到任何 Endpoint（manifest-buggy/ 的除錯題就是在考這個）
<u>__FILL_ME_23__</u> 08-planka-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port spec.ports[0].targetPort 提示：對應到 Pod 上 named port 的名稱，不是數字
<u>__FILL_ME_24__</u> 09-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 parentRefs[0].name 提示：看 ../../shared-infra/ 底下那份共用資源叫什麼名字

實作：填空（ 5/5 ）

manifest-incomplete/ · 共 30 格（ 第 5/5 頁 ） · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_25__</u> 09-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例： <code><product>.nexai.org.com</code> ，這個 product 是什麼？也要跟 <code>__FILL_ME_17__</code> 填的值一致
<u>__FILL_ME_26__</u> 09-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 08-planka-service.yaml 裡 Service 對外開的是幾號 port（不是 containerPort）
<u>__FILL_ME_27__</u> 10-networkpolicy.yaml	設定「放行 Gateway 進來的流量」這條規則要保護哪些 Pod <code>allow-ingress-to-planka-from-gateway.spec.podSelector.matchLabels.app</code> 提示：和 07-planka-deployment.yaml 用同一組 label
<u>__FILL_ME_28__</u> 10-networkpolicy.yaml	設定「只放行 planka 連進來」這條規則要保護哪些 Pod <code>allow-ingress-to-postgres-from-planka.spec.podSelector.matchLabels.app</code> 提示：這條規則保護的是資料庫那一邊，該用哪個 app label？
<u>__FILL_ME_29__</u> 10-networkpolicy.yaml	設定「允許連到 postgres」這條 egress 規則要套用在哪些 Pod 上 <code>allow-egress-planka-to-postgres.spec.podSelector.matchLabels.app</code> 提示：這是一條 egress 規則，套用對象是「要連出去」的那一邊——是 planka 還是 postgres？
<u>__FILL_ME_30__</u> 10-networkpolicy.yaml	設定這條 egress 規則允許連到哪些 Pod <code>allow-egress-planka-to-postgres.spec.egress[0].to[0].podSelector.matchLabels.app</code> 提示：這是 planka 要連去的目的地，同一個 namespace 裡誰在監聽 5432？

實作：填空・解答

30 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	planka
2	00 · label product	planka
3	01 · quota cpu	"1"
4	01 · max.cpu	"1"
5	01 · min.memory	128Mi
6	02 · POSTGRES_PASSWORD	"TrainingPlanka123"
7	03 · clusterIP	None
8	03 · selector.app	postgres
9	04 · serviceName	postgres
10	04 · PGDATA	/var/lib/postgresql/data/pgdata

#	欄位	正解
11	04 · liveness port	postgres
12	04 · accessModes	ReadWriteOnce
13	04 · storageClass	rook-ceph-block
14	05 · DATABASE_URL	"postgresql://planka:TrainingPlanka123@postgres:5432/planka"
15	06 · accessModes	ReadWriteMany
16	07 · image	ghcr.io/plankanban/planka:latest
17	07 · BASE_URL	"http://planka.nexai.org.com"
18	07 · containerPort	1337
19	07 · readiness port	http
20	07 · drop	ALL

#	欄位	正解
21	07 · mountPath	/app/data
22	08 · selector.app	planka
23	08 · targetPort	http
24	09 · parentRefs	dev-gateway
25	09 · hostnames	"planka.nexai.org.com"
26	09 · backendRefs	80
27	10 · podSelector	planka
28	10 · podSelector	postgres
29	10 · podSelector	planka
30	10 · to podSelector	postgres

實作：藏錯

manifest-buggy/ · 藏了 5 個錯 · 每個檔案最多一個 · 先整套 apply，再從症狀往回追

1 postgres-0 起不來

postgres-0 遲遲不 Running 或反覆重啟，initdb 因為「目錄不是空的」拒絕啟動。

先查

```
kubectl logs postgres-0 -n planka
```

2 Endpoints 永遠是空的

planka Pod Running，透過 Gateway 連線得到 503 / connection refused。

先查

```
kubectl get endpoints planka -n planka
```

3 容器建立不起來

不是 Running 也不是 CrashLoopBackOff，Events 說找不到某個 Secret 欄位 (key)。

先查

```
kubectl describe pod -n planka
```

4 planka 連不到資料庫

兩個 Pod 都 Running、NetworkPolicy 也套上了，但 planka 連 postgres:5432 逾時。

先查

```
kubectl logs deploy/planka -n planka
```

5 Gateway 連不到網站

describe httproute 顯示 Accepted: True，但透過 Gateway 發流量會失敗。

先查

```
kubectl describe httproute planka -n planka
```

除錯順序

1. get events：最近發生了什麼
2. describe / logs：找第一手線索
3. exec 進 Pod 測 DNS 與連線
4. 最後才 diff ../manifest/ 對答案

實作：藏錯・解答

5 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	postgres-0 起不來	04-postgres-statefulset.yaml	錯 PGDATA: /var/lib/postgresql/data 改 PGDATA: .../data/pgdata	ext4 掛載點根目錄自帶 lost+found，指到子目錄才是空目錄。
2	Endpoints 永遠是空的	08-planka-service.yaml	錯 selector.app: plankapp 改 selector.app: planka	selector 必須和 Pod label 一字不差。
3	容器建立不起來	07-planka-deployment.yaml	錯 secretKeyRef.key: DATABASE_URI 改 secretKeyRef.key: DATABASE_URL	key 拼錯會卡在 CreateContainerConfigError，容器根本不會啟動。
4	planka 連不到資料庫	10-networkpolicy.yaml	錯 from podSelector app: plankapp 改 from podSelector app: planka	postgres 的 ingress 只放行 label 對得上的來源 Pod。
5	Gateway 連不到網站	09-httproute.yaml	錯 backendRefs port: 1337 改 backendRefs port: 80	1337 是 containerPort；backendRefs 要填 Service 的 port。

第 3、4 站・對照

項目	flarum	planka
資料庫	MySQL 8.0 StatefulSet	PostgreSQL 16 StatefulSet
映像使用者	root 啟動再降權	非 root (uid 1000)
drop ALL	不行，會啟動失敗	可以
initContainer	需要，先建目錄與權限	不需要
更新策略	Recreate (RWO PVC)	RollingUpdate (RWX PVC)
DB 連線設定	DB_HOST / DB_USER / DB_PASS	DATABASE_URL 單一字串
NetworkPolicy	只有 app=flarum 能連 3306	只有 app=planka 能連 5432

第 5 站 · onlyoffice

重量級工作負載

是強大的線上 Office 辦公套件，支援 Word、Excel、PPT 的線上即時共同編輯，可與 Nextcloud、Filebrowser 等系統整合，官方提供「單容器全包」映像（`onlyoffice/documentserver`），內建自己的 PostgreSQL、Redis、RabbitMQ，全部包在同一個 container 裡執行，不需要外接資料庫。



實作：填空（1/4）

manifest-incomplete/ · 共 25 格（第 1/4 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_1__</u> 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示： 這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
<u>__FILL_ME_2__</u> 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示： 跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
<u>__FILL_ME_3__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示： Deployment 只有 1 個 replica、requests.cpu: "1"，但同檔案的 pods: "3" 代表 namespace 允許額外的臨時 Pod，配額要留這個空間；詳見上方「資源該給多少」章節第 4 點
<u>__FILL_ME_4__</u> 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU（上限） <code>LimitRange.max.cpu</code> 提示： 必須 ≥ Deployment 裡容器實際填的 limits.cpu；本產品是官方建議 2 CPU/4GB 起跳的重量級工作負載，上限該抓多少？
<u>__FILL_ME_5__</u> 01-resourcequota-limitrange.yaml	設定單一容器最少要 request 多少記憶體（下限） <code>LimitRange.min.memory</code> 提示： 必須 ≤ Deployment 裡容器實際填的 requests.memory，否則 Pod 會被 LimitRange 擋掉（下限擋掉輕量容器是本教材真的踩過的坑）
<u>__FILL_ME_6__</u> 02-secret.yaml	設定 JWT 簽章金鑰的值 <code>stringData.JWT_SECRET</code> 提示： 教學用途沒有格式限制，自己取一個字串即可，但要記得這把金鑰之後串接 filebrowser 時對方也要用同一把簽 token

實作：填空（2/4）

manifest-incomplete/ · 共 25 格（第 2/4 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_7__ 03-pvc.yaml	設定這個 PVC 要用哪一個 StorageClass <code>spec.storageClassName</code> 提示： 這座叢集沒有 default StorageClass，必須明確指定；這個 PVC 是單寫（ReadWriteOnce）場景，該挑哪一種？（提示：training.md 裡有列出這座叢集可用的 StorageClass 名稱）
__FILL_ME_8__ 03-pvc.yaml	設定要申請多少儲存容量 <code>spec.resources.requests.storage</code> 提示： 檔案上方教學註解已經寫出實際要申請的容量
__FILL_ME_9__ 04-deployment.yaml	設定要把 Pod 排到哪一台 Node 上 <code>spec.template.spec.nodeSelector."kubernetes.io/hostname"</code> 提示： 檔案上方教學註解說瞭這座叢集哪一台 Node 記憶體用量最低、適合塞下這個吃重的工作負載（此行預設已註解、改以 toleration 為主不指定節點；屬選填，若要練習請取消註解後再填）
__FILL_ME_10__ 04-deployment.yaml	設定要容忍（tolerate）哪一個 taint 的 key <code>tolerations[0].key</code> 提示： 目標 Node 身上帶的是哪個廠牌的 GPU taint？（提示：nvidia.com/...）
__FILL_ME_11__ 04-deployment.yaml	設定要容忍的 taint value 要等於什麼 <code>tolerations[0].value</code> 提示： 對照這台 Node 身上 taint 的實際 value（在 kubectl describe node gpu01 看得到）
__FILL_ME_12__ 04-deployment.yaml	設定要容忍的 taint effect 是什麼 <code>tolerations[0].effect</code> 提示： taint 的三種 effect（NoSchedule/PreferNoSchedule/NoExecute）中，這座叢集的 GPU taint 用的是哪一種？

實作：填空（3/4）

manifest-incomplete/ · 共 25 格（第 3/4 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_13__</u> 04-deployment.yaml	填入 ONLYOFFICE 官方容器映像的名稱與版本 <code>containers[0].image</code> 提示：ONLYOFFICE Document Server 官方 Docker Hub 映像名稱，這裡用的是哪個 tag？
<u>__FILL_ME_14__</u> 04-deployment.yaml	設定是否啟用外掛管理背景程序 <code>env[PLUGINS_ENABLED].value</code> 提示：檔案上方教學註解整段都在講這個環境變數為什麼要關掉，仔細讀完再填
<u>__FILL_ME_15__</u> 04-deployment.yaml	填入容器內部應用程式實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示：跟 Service 的 targetPort 這個 named port 要對得上
<u>__FILL_ME_16__</u> 04-deployment.yaml	設定 readiness 探測要打哪個路徑 <code>readinessProbe.httpGet.path</code> 提示：ONLYOFFICE Document Server 官方內建的健康檢查端點叫什麼名字？（跟 livenessProbe 那組保持一致寫法）
<u>__FILL_ME_17__</u> 04-deployment.yaml	設定 liveness 探測要打哪個 port <code>livenessProbe.httpGet.port</code> 提示：可以直接引用上面 ports 陣列裡取的 name，不用重複寫數字
<u>__FILL_ME_18__</u> 04-deployment.yaml	設定資料要掛到容器裡的哪個路徑 <code>volumeMounts[0](data).mountPath</code> 提示：檔案上方教學註解明確寫出了這個「有明確文件、單純的資料路徑」是什麼
<u>__FILL_ME_19__</u> 05-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示：跟 Deployment 的 template.metadata.labels.app 必須完全一致，否則 Service 找不到任何 Endpoint（manifest-buggy/ 的除錯題就是在考這個）

實作：填空（4/4）

manifest-incomplete/ · 共 25 格（第 4/4 頁）· 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_20__</u> 05-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱，不是數字
<u>__FILL_ME_21__</u> 06-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[0].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字
<u>__FILL_ME_22__</u> 06-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例：<product>.nexai.org.com，這個 product 是什麼？
<u>__FILL_ME_23__</u> 06-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 05-service.yaml 裡 Service 對外開的是幾號 port（不是 containerPort）
<u>__FILL_ME_24__</u> 07-networkpolicy.yaml	設定 allow-ingress-from-gateway 要保護哪些 Pod <code>allow-ingress-from-gateway.spec.podSelector.matchLabels.app</code> 提示：和 Service/Deployment 用同一組 label
<u>__FILL_ME_25__</u> 07-networkpolicy.yaml	設定要放行到哪一個 namespace 的出向流量 <code>allow-egress-onlyoffice-to-filebrowser...namespaceSelector.matchLabels."kubernetes.io/metadata.name"</code> 提示：檔案上方教學註解說明了 Document Server 要主動回頭連哪個系統抓檔案內容

實作：填空・解答

25 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	onlyoffice
2	00 · label product	onlyoffice
3	01 · quota cpu	"2"
4	01 · max.cpu	"4"
5	01 · min.memory	512Mi
6	02 · JWT_SECRET	"TrainingOnlyofficeJwtSecret_ChangeMe"
7	03 · storageClass	rook-ceph-block
8	03 · storage	5Gi
9	04 · nodeSelector	gpu01
10	04 · toleration key	nvidia.com/gpu
11	04 · toleration value	"true"
12	04 · toleration effect	NoSchedule
13	04 · image	onlyoffice/documentserver:latest

#	欄位	正解
14	04 · PLUGINS_ENABLED	"false"
15	04 · containerPort	80
16	04 · readiness path	/healthcheck
17	04 · liveness port	http
18	04 · mountPath	/var/www/onlyoffice/Data
19	05 · selector.app	onlyoffice
20	05 · targetPort	http
21	06 · parentRefs	dev-gateway
22	06 · hostnames	"onlyoffice.nexai.org.com"
23	06 · backendRefs	80
24	07 · podSelector	onlyoffice
25	07 · namespaceSelector	filebrowser

實作：藏錯

manifest-buggy/ · 藏了 4 個錯 · 每個檔案最多一個 · 先整套 apply · 再從症狀往回追

1 CPU 異常偏高

Running 1/1、功能也正常，但沒人使用時 CPU 仍長時間貼著不低的水位。

先查

```
kubectl top pod -n onlyoffice
```

2 PVC 一直卡在 Pending

Pod 跟著卡在 Pending / ContainerCreating，Events 說找不到對應的 StorageClass。

先查

```
kubectl describe pvc onlyoffice-data -n onlyoffice
```

3 Endpoints 永遠是空的

Pod Running，透過 Gateway 連線得到 503 / connection refused。跟 label 有關。

先查

```
kubectl get endpoints onlyoffice -n onlyoffice
```

4 HTTPRoute Accepted 卻連不到

HTTPRoute 顯示 Accepted: True，實際流量卻失敗或連不到應用。

先查

```
kubectl describe httproute onlyoffice -n onlyoffice
```

實作：藏錯・解答

4 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	CPU 異常偏高	04-deployment.yaml	錯 (缺) <code>PLUGINS_ENABLED</code> 改 <code>PLUGINS_ENABLED: "false"</code>	外掛程序連外網被擋，卡在吃滿 1 核心的重試迴圈。
2	PVC 一直卡在 Pending	03-pvc.yaml	錯 <code>storageClassName: rook-ceph-blk</code> 改 <code>storageClassName: rook-ceph-block</code>	叢集沒有 default StorageClass，名字拼錯就永遠不會 provision。
3	Endpoints 永遠是空的	05-service.yaml	錯 <code>selector.app: onlyofice</code> 改 <code>selector.app: onlyoffice</code>	少一個 f，Service 就選不到任何 Pod。
4	HTTPRoute Accepted 卻連不到	06-httprouete.yaml	錯 <code>backendRefs port: 8080</code> 改 <code>backendRefs port: 80</code>	Service 對外開的是 80，這個 Service 上根本沒有 8080。

第 6 站 · peertube

三個元件一起協作 (PostgreSQL, Redis, PeerTube)

是基於 ActivityPub 協定與 P2P 技術的 去中心化影音分享平台 (開源版 YouTube) 。
(chocobozzz/peertube 映像)



實作：填空（1/7）

manifest-incomplete/ · 共 43 格（第 1/7 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_1__</u> 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示： 這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
<u>__FILL_ME_2__</u> 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示： 跟 __FILL_ME_1__ 填同一個值即可（本教材慣例：label 值 = namespace 名稱）
<u>__FILL_ME_3__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示： 這個 namespace 同時跑 postgres/redis/peertube 三個元件，要把三者的 requests.cpu 加總後再留餘裕，見上面「資源該給多少」章節的計算
<u>__FILL_ME_4__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 Memory 總量 <code>ResourceQuota.hard.requests.memory</code> 提示： 同上，三個元件的 requests.memory 加總後留餘裕
<u>__FILL_ME_5__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能建立幾個 PVC <code>ResourceQuota.hard.persistentvolumeclaims</code> 提示： 這個 namespace 實際會建立幾個 PVC？（提示：不是只有 postgres 有，peertube 本體也有一個）
<u>__FILL_ME_6__</u> 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU（上限） <code>LimitRange.max.cpu</code> 提示： 必須 ≥ 三個元件裡 limits.cpu 最大的那一個，是哪個元件？

實作：填空（2/7）

manifest-incomplete/ · 共 43 格（第 2/7 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_7__</u> 01-resourcequota-limitrange.yaml	設定單一容器最少要 request 多少記憶體（下限） <code>LimitRange.min.memory</code> 提示：必須 $\leq$ 三個元件裡 <code>requests.memory</code> 最小的那一個，是哪個元件？
<u>__FILL_ME_8__</u> 02-postgres-secret.yaml	設定 PostgreSQL 的登入帳號 <code>stringData.POSTGRES_USER</code> 提示：這個值之後會被 09-peertube-deployment.yaml 的 <code>PEERTUBE_DB_USERNAME</code> 引用，兩邊要對得上
<u>__FILL_ME_9__</u> 02-postgres-secret.yaml	設定 PostgreSQL 要建立的資料庫名稱 <code>stringData.POSTGRES_DB</code> 提示：這個值之後會被 09-peertube-deployment.yaml 的 <code>PEERTUBE_DB_NAME</code> 引用，兩邊要對得上
<u>__FILL_ME_10__</u> 03-postgres-initdb-configmap.yaml	幫初始化腳本取檔名 <code>data.<key></code> （檔名） 提示：官方 postgres image 只認 <code>/docker-entrypoint-initdb.d/</code> 底下副檔名 <code>.sql</code> / <code>.sh</code> 的檔案，檔名本身沒有強制規則，但要讓人一看就懂用途
<u>__FILL_ME_11__</u> 03-postgres-initdb-configmap.yaml	填入 PeerTube 官方文件要求的第一個 PostgreSQL extension <code>SQL 裡的 extension 名稱</code> 提示：做「模糊搜尋」用的那個 extension，README 說明段落有提到
<u>__FILL_ME_12__</u> 03-postgres-initdb-configmap.yaml	填入 PeerTube 官方文件要求的第二個 PostgreSQL extension <code>SQL 裡的 extension 名稱</code> 提示：做「去重音搜尋」用的那個 extension，README 說明段落有提到

實作：填空 (3/7)

manifest-incomplete/ · 共 43 格 (第 3/7 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_13__</u> 04-postgres-service.yaml	設定這個 Service 要不要配發 ClusterIP spec.clusterIP 提示： StatefulSet 通常搭配哪一種 Service (讓每個 Pod 有自己的 DNS 記錄，而不是共用一個負載平衡 IP) ？填哪個特殊值代表「不要配發」？
<u>__FILL_ME_14__</u> 04-postgres-service.yaml	設定這個 Service 要選中哪些 Pod spec.selector.app 提示： 跟 05-postgres-statefulset.yaml 的 template.metadata.labels.app 必須完全一致
<u>__FILL_ME_15__</u> 04-postgres-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port spec.ports[0].targetPort 提示： 對應到 Pod 上 named port 的名稱，不是數字
<u>__FILL_ME_16__</u> 05-postgres-statefulset.yaml	設定要把 Pod 排到哪一個節點 spec.template.spec.nodeSelector.<key> (節點名稱) 提示： 教學重點段落說明了為什麼這三個元件都要排到同一個節點，是哪個節點？ (此行預設已註解、改以 toleration 為主不指定節點；屬選填，若要練習請取消註解後再填)
<u>__FILL_ME_17__</u> 05-postgres-statefulset.yaml	設定 initdb 腳本要掛到容器裡的哪個路徑 volumeMounts[1].mountPath 提示： 官方 postgres image 只會自動執行這一個固定路徑底下的初始化腳本，路徑錯了 extension 永遠不會被建立 (可對照 manifest-buggy/ 的除錯題感受這個症狀)
<u>__FILL_ME_18__</u> 05-postgres-statefulset.yaml	設定 initdb volume 要引用哪個 ConfigMap volumes[0].configMap.name 提示： 要跟 03-postgres-initdb-configmap.yaml 的 metadata.name 一致

實作：填空（4/7）

manifest-incomplete/ · 共 43 格（第 4/7 頁）· 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_19__</u> 05-postgres-statefulset.yaml	設定 PVC 要用哪個 StorageClass <code>volumeClaimTemplates[0].spec.storageClassName</code> 提示：這座叢集沒有 default StorageClass，資料庫類單寫場景該選 RWO 還是 RWX 的那個 class？（../README.md 或 ../training.md 有列出叢集的 StorageClass 名稱）
<u>__FILL_ME_20__</u> 05-postgres-statefulset.yaml	設定 postgres 資料需要多少儲存空間 <code>volumeClaimTemplates[0].spec.resources.requests.storage</code> 提示：這是教學用途的小容量配置，抓個位數 Gi 即可
<u>__FILL_ME_21__</u> 06-redis.yaml	設定 redis Service 要選中哪些 Pod <code>Service.spec.selector.app</code> 提示：跟同檔案 Deployment 的 <code>template.metadata.labels.app</code> 必須完全一致（manifest-buggy/ 的除錯題就是在考這個）
<u>__FILL_ME_22__</u> 06-redis.yaml	設定 redis Service 要把流量轉去 Pod 的哪個 port <code>Service.spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱，不是數字
<u>__FILL_ME_23__</u> 06-redis.yaml	設定要把 redis Pod 排到哪一個節點 <code>Deployment.spec.template.spec.nodeSelector.<key></code> （節點名稱） 提示：跟 __FILL_ME_16__ 應該填同一個節點名稱——三個元件為什麼要排到同一台？（此行預設已註解、改以 toleration 為主不指定節點；屬選填，若要練習請取消註解後再填）
<u>__FILL_ME_24__</u> 06-redis.yaml	填入 redis 容器內部實際監聽的 port <code>Deployment...containers[0].ports[0].containerPort</code> 提示：redis 的預設監聽埠是多少？要跟 __FILL_ME_22__ 對得上

實作：填空（ 5/7 ）

manifest-incomplete/ · 共 43 格（ 第 5/7 頁 ） · 照編號填完再依序 apply，最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_25__</u> 07-peertube-secret.yaml	設定管理員帳號的 Email <code>stringData.PEERTUBE_ADMIN_EMAIL</code> 提示： 這是教學用途的固定值，只有在資料庫全新初始化時才會生效，任意填一個看起來像 email 的字串即可
<u>__FILL_ME_26__</u> 07-peertube-secret.yaml	設定 PeerTube 應用程式連線資料庫要用的密碼 <code>stringData.PEERTUBE_DB_PASSWORD</code> 提示： 這個值必須跟 02-postgres-secret.yaml 的 POSTGRES_PASSWORD 完全一致，否則 PeerTube 連得到 postgres 這個位址、但認證會失敗（ manifest-buggy/ 的除錯題就是在考這個 ）
<u>__FILL_ME_27__</u> 08-peertube-pvc.yaml	設定 PVC 要用哪個 StorageClass <code>spec.storageClassName</code> 提示： 跟 __FILL_ME_19__ 填法邏輯一樣，這座叢集沒有 default StorageClass
<u>__FILL_ME_28__</u> 08-peertube-pvc.yaml	設定這個 PVC 的存取模式 <code>spec.accessModes[0]</code> 提示： 檔案上方教學註解解釋了為什麼這裡選單寫而不是多寫共享，該填哪個存取模式？
<u>__FILL_ME_29__</u> 09-peertube-deployment.yaml	設定「使用者實際存取網址」用的埠號 <code>env[PEERTUBE_WEBSERVER_PORT].value</code> 提示： 這裡要填的是瀏覽器打進來看到的埠號（ 透過 Gateway 走 https ），不是容器內部監聽的埠號——兩者是完全不同的兩件事，檔案上方教學註解有詳細解釋
<u>__FILL_ME_30__</u> 09-peertube-deployment.yaml	設定「使用者實際存取網址」是否為 https <code>env[PEERTUBE_WEBSERVER_HTTPS].value</code> 提示： 這個產品的 HTTPRoute 只掛了 https 監聽器，這裡該填 "true" 還是 "false" ？

實作：填空 (6/7)

manifest-incomplete/ · 共 43 格 (第 6/7 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_31__ 09-peertube-deployment.yaml	設定要信任哪些來源的反向代理位址 <code>env[PEERTUBE_TRUST_PROXY].value</code> 提示： PeerTube 預設只信任 loopback (127.0.0.1) · 但流量是從 Gateway 的 Pod 網路進來的，要額外加上哪個值才能讓 PeerTube 正確判斷來源 IP / 協定？ (提示：RFC1918 私有位址的意思)
__FILL_ME_32__ 09-peertube-deployment.yaml	設定要連去哪個 hostname 存取 Redis <code>env[PEERTUBE_REDIS_HOSTNAME].value</code> 提示： 要跟 06-redis.yaml 的 Service metadata.name 一致
__FILL_ME_33__ 09-peertube-deployment.yaml	設定要連去哪個 hostname 存取 PostgreSQL <code>env[PEERTUBE_DB_HOSTNAME].value</code> 提示： 要跟 04-postgres-service.yaml 的 Service metadata.name 一致
__FILL_ME_34__ 09-peertube-deployment.yaml	填入 PeerTube 容器內部實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示： 這是容器自己的監聽埠，注意跟 __FILL_ME_29__ 填的「外部埠號」是不同層次的兩個數字，不要填成一樣
__FILL_ME_35__ 09-peertube-deployment.yaml	設定要掛載哪個 PVC 當作影片儲存空間 <code>volumes[0].persistentVolumeClaim.claimName</code> 提示： 要跟 08-peertube-pvc.yaml 的 metadata.name 一致
__FILL_ME_36__ 10-peertube-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示： 跟 09-peertube-deployment.yaml 的 template.metadata.labels.app 必須完全一致
__FILL_ME_37__ 10-peertube-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示： 對應到 Pod 上 named port 的名稱，不是數字

實作：填空（ 7/7 ）

manifest-incomplete/ · 共 43 格 (第 7/7 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_38__</u> 11-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例：<product>.nexai.org.com · 這個 product 是什麼？
<u>__FILL_ME_39__</u> 11-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 10-peertube-service.yaml 裡 Service 對外開的是幾號 port (不是 containerPort · 也不是 __FILL_ME_29__ 那個外部埠號)
<u>__FILL_ME_40__</u> 11-httproute.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[0].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字
<u>__FILL_ME_41__</u> 12-networkpolicy.yaml	設定只放行進到 peertube 容器的哪個 port <code>allow-ingress-to-peertube-from-gateway.spec.ingress[0].ports[0].port</code> 提示：跟 __FILL_ME_34__ 應該是同一個號碼 (NetworkPolicy 管的是實際監聽的容器 port · 不是外部埠號)
<u>__FILL_ME_42__</u> 12-networkpolicy.yaml	設定這條規則要保護哪個元件的 Pod <code>allow-ingress-to-postgres-from-peertube.spec.podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你它在保護誰
<u>__FILL_ME_43__</u> 12-networkpolicy.yaml	設定要放行「哪個元件」打進來的流量到 redis <code>allow-ingress-to-redis-from-peertube.spec.ingress[0].from[0].podSelector.matchLabels.app</code> 提示：這條規則的名字已經告訴你放行的來源是誰

實作：填空・解答

43 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	peertube
2	00 · label product	peertube
3	01 · quota cpu	"2"
4	01 · quota memory	3Gi
5	01 · quota PVCs	"4"
6	01 · max.cpu	"2"
7	01 · min.memory	64Mi
8	02 · POSTGRES_USER	"peertube"
9	02 · POSTGRES_DB	"peertube"
10	03 · ConfigMap key	init-extensions.sql
11	03 · EXTENSION	pg_trgm
12	03 · EXTENSION	unaccent
13	04 · clusterIP	None
14	04 · selector.app	postgres
15	04 · targetPort	postgres

#	欄位	正解
16	05 · nodeSelector	gpu01
17	05 · mountPath	/docker-entrypoint-initdb.d
18	05 · configMap	postgres-initdb
19	05 · storageClass	rook-ceph-block
20	05 · storage	5Gi
21	06 · selector.app	redis
22	06 · targetPort	redis
23	06 · nodeSelector	gpu01
24	06 · containerPort	6379
25	07 · ADMIN_EMAIL	"admin@example.com"
26	07 · DB_PASSWORD	"TrainingPeertube123"
27	08 · storageClass	rook-ceph-block
28	08 · accessModes	ReadWriteOnce
29	09 · WEBSERVER_PORT	"443"
30	09 · WEBSERVER_HTTPS	"true"

#	欄位	正解
31	09 · TRUST_PROXY	'["loopback","linklocal","uniquelocal"]'
32	09 · REDIS_HOSTNAME	"redis"
33	09 · DB_HOSTNAME	"postgres"
34	09 · containerPort	9000
35	09 · claimName	peertube-storage
36	10 · selector.app	peertube
37	10 · targetPort	http
38	11 · hostnames	"peertube.nexai.org.com"
39	11 · backendRefs	80
40	11 · parentRefs	dev-gateway
41	12 · ingress port	9000
42	12 · podSelector	postgres
43	12 · from podSelector	peertube

實作：藏錯

manifest-buggy/ · 藏了 6 個錯 · 每個檔案最多一個 · 先整套 apply · 再從症狀往回追

1 資料庫認證失敗

PeerTube Running 且 Ready · 註冊或新增影片時 logs 出現 authentication failed 。

先查

```
kubectl logs deploy/peertube -n peertube
```

2 來源 IP / https 判斷異常

網站打得開、一切看似正常，但記錄的來源 IP 或 https 判斷不對，很容易被忽略。

先查

```
kubectl logs deploy/peertube -n peertube
```

3 migration 缺 extension

postgres-0 健康，但 PeerTube 第一次跑 migration 就失敗，提到缺少 extension 。

先查

```
kubectl logs postgres-0 -n peertube
```

4 redis 的 Endpoints 是空的

redis Pod Ready、ping 也正常，但 PeerTube 連不到 redis，佇列功能出錯。

先查

```
kubectl get endpoints redis -n peertube
```

5 套用 NetworkPolicy 後逾時

套用前一切正常，套用後 PeerTube 連 postgres timeout (不是認證失敗) 。

先查

```
kubectl get networkpolicy -n peertube
```

6 Gateway 連不到網站

HTTPRoute 顯示 Accepted: True，直接測 Service 正常，透過 Gateway 卻失敗。

先查

```
kubectl describe httproute -n peertube
```

實作：藏錯・解答

6 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	資料庫認證失敗	07-peertube-secret.yaml	錯 DB_PASSWORD: "TrainingPeertubeApp123" 改 DB_PASSWORD: "TrainingPeertube123"	要跟 02-postgres-secret 的 POSTGRES_PASSWORD 完全一致。
2	來源 IP / https 判斷異常	09-peertube-deployment.yaml	錯 TRUST_PROXY: 少了 "uniquelocal" 改 ["loopback", "linklocal", "uniquelocal"]	Gateway 從 Pod 網路 (私有位址) 轉進來，要信任 uniquelocal。
3	migration 缺 extension	05-postgres-statefulset.yaml	錯 ../docker-entrypoint-initdb.d/extensions 改 /docker-entrypoint-initdb.d	官方映像只執行這個目錄第一層的 .sql / .sh，子目錄不會跑。
4	redis 的 Endpoints 是空的	06-redis.yaml	錯 selector.app: redis-cache 改 selector.app: redis	selector 必須和 redis Pod 的 label 一字不差。
5	套用 NetworkPolicy 後逾時	12-networkpolicy.yaml	錯 postgres ingress port: 5433 改 postgres ingress port: 5432	放行錯的 port 等於沒放行，封包被悄悄丟棄才會是 timeout。
6	Gateway 連不到網站	11-httproute.yaml	錯 backendRefs port: 9000 改 backendRefs port: 80	9000 是 containerPort；Service 對外開的是 80。

第 7 站 · cloudbeaver

唯一的跨 namespace 存取

是 DBeaver 官方推出的 Web 版本，同一個 瀏覽器介面可以建立多組資料庫連線、支援 MySQL/PostgreSQL 等多種資料庫，是一個 通用的視覺化資料庫管理工具（DBeaver-like）。

（ dbeaver/cloudbeaver 映像 ）



實作：填空 (1/5)

manifest-incomplete/ · 共 28 格 (第 1/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_1__</u> 00-namespace.yaml	幫這個 namespace 取名字 <code>metadata.name</code> 提示： 這個值之後每個檔案的 namespace: 都要跟它一致，README 標題已經告訴你這個產品叫什麼
<u>__FILL_ME_2__</u> 00-namespace.yaml	幫這個 namespace 加上分類標籤 <code>labels.training/product</code> 提示： 跟 __FILL_ME_1__ 填同一個值即可 (本教材慣例：label 值 = namespace 名稱)
<u>__FILL_ME_3__</u> 01-resourcequota-limitrange.yaml	設定整個 namespace 最多能同時 request 多少 CPU 總量 <code>ResourceQuota.hard.requests.cpu</code> 提示： Deployment 只開 1 個 replica，容器 requests.cpu: 250m；配額至少要蓋過這個量，並留一點給未來可能加的第二個資料庫周邊工具 (pods: "3" 已經暗示留了名額) (進階提示：填得太小會讓唯一的 Pod 直接被 quota 擋在門外，連 Pending 都排不進去，可對照 manifest-buggy/ 的除錯題感受這個症狀)
<u>__FILL_ME_4__</u> 01-resourcequota-limitrange.yaml	設定單一容器最多能要多少 CPU (上限) <code>LimitRange.max.cpu</code> 提示： 必須 ≥ Deployment 裡容器實際填的 limits.cpu，否則 Pod 會被 LimitRange 擋掉
<u>__FILL_ME_5__</u> 01-resourcequota-limitrange.yaml	設定單一容器最少要 request 多少記憶體 (下限) <code>LimitRange.min.memory</code> 提示： 必須 ≤ Deployment 裡容器實際填的 requests.memory，抓太高會擋掉未來想加進這個 namespace 的輕量容器 (這門課幫 cloudbeaver 加 Adminer 時真的踩過這個坑)

實作：填空 (2/5)

manifest-incomplete/ · 共 28 格 (第 2/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_6__ 02-pvc.yaml	設定這個 PVC 的存取模式 <code>spec.accessModes[0]</code> 提示：只有 1 個 replica 會掛這顆 PVC，需要「單一節點可讀寫」還是「多節點共享讀寫」？（跟 filebrowser 那種多 Pod 共享的情境不一樣）
__FILL_ME_7__ 02-pvc.yaml	設定要用哪個 StorageClass <code>spec.storageClassName</code> 提示：這座叢集沒有 default StorageClass，要明確指定；RWO 單寫場景這門課統一用哪個 Ceph StorageClass？（可參考其他資料庫類產品，例如 flarum/planka 的 PVC）
__FILL_ME_8__ 02-pvc.yaml	設定要申請多少容量 <code>spec.resources.requests.storage</code> 提示：只是存帳號/連線設定的 workspace 目錄，不是存實際資料表內容，不需要很大
__FILL_ME_9__ 03-deployment.yaml	填入 CloudBeaver 官方容器映像的名稱 <code>containers[0].image</code> 提示：DBeaver 官方在 Docker Hub 發布的 CloudBeaver 映像叫什麼？檔案上方教學註解與 README 產品說明都有寫
__FILL_ME_10__ 03-deployment.yaml	填入容器內部應用程式實際監聽的 port <code>containers[0].ports[0].containerPort</code> 提示：跟 04-service.yaml 的 targetPort: http 這個 named port、以及 06-networkpolicy.yaml 放行 Gateway ingress 的那個 port 要三邊對得上
__FILL_ME_11__ 03-deployment.yaml	設定 liveness 探測要等待幾秒才開始檢查 <code>livenessProbe.initialDelaySeconds</code> 提示：檔案上方教學註解直接寫了原因：Java 應用冷啟動比 PHP/Node 慢，太早開始檢查會在應用程式還在初始化時就被砍掉；這個值該抓多寬鬆？（比 readinessProbe 的 30 秒還要更長）

實作：填空（ 3/5 ）

manifest-incomplete/ · 共 28 格（ 第 3/5 頁 ） · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_12__</u> 03-deployment.yaml	設定 workspace 要掛載到容器內的哪個路徑 <code>volumeMounts[0].mountPath</code> 提示：CloudBeaver 的 GlobalConfigurationDir · 檔案上方教學註解有寫這個目錄的用途
<u>__FILL_ME_13__</u> 03-deployment.yaml	設定要掛哪一個 PVC <code>volumes[0].persistentVolumeClaim.claimName</code> 提示：要跟 02-pvc.yaml 裡 metadata.name 完全一致，否則 Pod 會卡在找不到 PVC
<u>__FILL_ME_14__</u> 03-deployment.yaml	觀念題（不是 YAML 值）：這個 container 為什麼沒有 <code>securityContext.capabilities.drop</code> 無對應欄位，寫在旁邊或口頭回答即可 提示：對照 ../draw.io/manifest/02-deployment.yaml 的 <code>capabilities.drop: [ALL]</code> ——CloudBeaver 官方 entrypoint 用 root 身分做了什麼事，才需要保留預設 capability？（提示：跟 mysql/postgres/redis 同一種「先 chown PVC 再降權」模式）
<u>__FILL_ME_15__</u> 04-service.yaml	設定這個 Service 要選中哪些 Pod <code>spec.selector.app</code> 提示：跟 Deployment 的 <code>template.metadata.labels.app</code> 必須完全一致，否則 Service 找不到任何 Endpoint（manifest-buggy/ 的除錯題就是在考這個）
<u>__FILL_ME_16__</u> 04-service.yaml	設定 Service 要把流量轉去 Pod 的哪個 port <code>spec.ports[0].targetPort</code> 提示：對應到 Pod 上 named port 的名稱，不是數字
<u>__FILL_ME_17__</u> 05-httprouete.yaml	設定這個 HTTPRoute 要掛在哪個 Gateway 底下 <code>parentRefs[0].name</code> 提示：看 ../shared-infra/ 底下那份共用資源叫什麼名字

實作：填空（4/5）

manifest-incomplete/ · 共 28 格（第 4/5 頁） · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
<u>__FILL_ME_18__</u> 05-httproute.yaml	設定對外存取這個服務要用的網域名稱 <code>hostnames[0]</code> 提示：沿用叢集網域慣例： <code><product>.nexai.org.com</code> · 這個 product 是什麼？
<u>__FILL_ME_19__</u> 05-httproute.yaml	設定 HTTPRoute 要把流量轉去 Service 的哪個 port <code>backendRefs[0].port</code> 提示：看 04-service.yaml 裡 Service 對外開的是幾號 port（不是 containerPort）
<u>__FILL_ME_20__</u> 06-networkpolicy.yaml	設定「放行 Gateway 進來」這條規則要保護哪些 Pod <code>allow-ingress-to-cloudbeaver-from-gateway.spec.podSelector.matchLabels.app</code> 提示：和 Service/Deployment 用同一組 label
<u>__FILL_ME_21__</u> 06-networkpolicy.yaml	設定只放行進到容器的哪個 port <code>allow-ingress-to-cloudbeaver-from-gateway.spec.ingress[0].ports[0].port</code> 提示：跟 <u>__FILL_ME_10__</u> 應該是同一個號碼
<u>__FILL_ME_22__</u> 06-networkpolicy.yaml	設定「連到三個資料庫」這條 egress 規則要套用在哪些 Pod 上 <code>allow-egress-cloudbeaver-to-databases.spec.podSelector.matchLabels.app</code> 提示：一樣是這個產品的 Pod · 和上面幾格填同一個值

實作：填空 (5/5)

manifest-incomplete/ · 共 28 格 (第 5/5 頁) · 照編號填完再依序 apply · 最後 diff -u 對照 ../manifest/

編號 / 檔案	題目
__FILL_ME_23__ 06-networkpolicy.yaml	設定第一條 egress 規則要放行連到哪個 namespace (flarum) <code>egress[0].to[0].namespaceSelector.matchLabels."kubernetes.io/metadata.name"</code> 提示： 這是這個產品的核心教學重點：不是用自訂 label，是用 K8s 幫每個 namespace 自動加上的內建標籤，值就是 namespace 的名字本身。這條要放行去 flarum
__FILL_ME_24__ 06-networkpolicy.yaml	設定第一條 egress 規則只放行到哪個 port <code>egress[0].ports[0].port</code> 提示： flarum 那個 mysql 監聽在哪个標準 port ? README 上方的資料庫對照表有寫
__FILL_ME_25__ 06-networkpolicy.yaml	設定第二條 egress 規則要放行連到哪個 namespace (planka) <code>egress[1].to[0].namespaceSelector.matchLabels."kubernetes.io/metadata.name"</code> 提示： 同 __FILL_ME_23__ 的邏輯，這條要放行去 planka
__FILL_ME_26__ 06-networkpolicy.yaml	設定第二條 egress 規則只放行到哪個 port <code>egress[1].ports[0].port</code> 提示： planka 那個 postgres 監聽在哪个標準 port ?
__FILL_ME_27__ 06-networkpolicy.yaml	設定第三條 egress 規則要放行連到哪個 namespace (peertube) <code>egress[2].to[0].namespaceSelector.matchLabels."kubernetes.io/metadata.name"</code> 提示： 同上，這條要放行去 peertube
__FILL_ME_28__ 06-networkpolicy.yaml	設定第三條 egress 規則只放行到哪個 port <code>egress[2].ports[0].port</code> 提示： peertube 那個 postgres 監聽在哪个標準 port ?

實作：填空・解答

28 格正解・標準答案就是 ../manifest/ 同名檔案・欄位前的數字是檔案編號

#	欄位	正解
1	00 · Namespace name	cloudbeaver
2	00 · label product	cloudbeaver
3	01 · quota cpu	"500m"
4	01 · max.cpu	"2"
5	01 · min.memory	256Mi
6	02 · accessModes	ReadWriteOnce
7	02 · storageClass	rook-ceph-block
8	02 · storage	2Gi
9	03 · image	dbeaver/cloudbeaver:latest
10	03 · containerPort	8978
11	03 · liveness delay	90
12	03 · mountPath	/opt/cloudbeaver/workspace
13	03 · claimName	cloudbeaver-workspace
14	03 · 觀念題	不設 drop (需 root chown)

#	欄位	正解
15	04 · selector.app	cloudbeaver
16	04 · targetPort	http
17	05 · parentRefs	dev-gateway
18	05 · hostnames	"cloudbeaver.nexai.org.com"
19	05 · backendRefs	80
20	06 · podSelector	cloudbeaver
21	06 · ingress port	8978
22	06 · podSelector	cloudbeaver
23	06 · namespaceSelector	flarum
24	06 · egress port	3306
25	06 · namespaceSelector	planka
26	06 · egress port	5432
27	06 · namespaceSelector	peertube
28	06 · egress port	5432

實作：藏錯

manifest-buggy/ · 藏了 4 個錯 · 每個檔案最多一個 · 先整套 apply · 再從症狀往回追

1 連第一個 Pod 都生不出來

只有 1 個 replica · Deployment 卻建不出任何 Pod · Events 看到 exceeded quota 。

先查

```
kubectl get events -n cloudbeaver
```

2 CrashLoopBackOff : permission

容器一起來就死 · logs 出現權限相關錯誤 · 官方 entrypoint 需不需要 root 去動 PVC ?

先查

```
kubectl logs deploy/cloudbeaver -n cloudbeaver --previous
```

3 Endpoints 永遠是空的

Pod Running 且 1/1 Ready · 透過 Gateway 連線得到 503 / connection refused 。

先查

```
kubectl get endpoints cloudbeaver -n cloudbeaver
```

4 三個 DB 只連得到兩個

在 Pod 裡用 /dev/tcp 測試 · 第三個資料庫卡住逾時 (不是立刻被拒絕) 。

先查

```
kubectl exec -it deploy/cloudbeaver -n cloudbeaver -- sh
```

實作：藏錯・解答

4 個錯・修完用四個綠燈驗收：Pod Ready、有 endpoints、HTTPRoute Accepted + ResolvedRefs、curl 拿到真內容

#	症狀	檔案	錯誤 → 修正	為什麼
1	連第一個 Pod 都生不出來	01-resourcequota-limitrange.yaml	錯 requests.cpu: "100m" 改 requests.cpu: "500m"	容器 requests.cpu 是 250m，配額 100m 讓 ReplicaSet 直接被拒。
2	CrashLoopBackOff : permission	03-deployment.yaml	錯 securityContext: drop [ALL] 改 移除這段 securityContext	entrypoint 先用 root chown PVC 再降權，drop ALL 會讓這一步失敗。
3	Endpoints 永遠是空的	04-service.yaml	錯 selector.app: cloudbeaver-app 改 selector.app: cloudbeaver	selector 必須和 Pod label 一字不差。
4	三個 DB 只連得到兩個	06-networkpolicy.yaml	錯 metadata.name: peertube-app 改 metadata.name: peertube	namespaceSelector 指到不存在的 namespace，等於沒放行。